National University
Of Computer and Emerging Sciences

Foundation of Advancement
Of Science and Technology

Lab Number:	03
Course Code:	CL2001
Course Title:	Data Structures - Lab
Semester:	Fall 2026
Instructor:	Muhammad Mehdi
Email:	muhammad.mehdi@nu.edu.pk

Guidelines

- You may use **only the concepts covered in the lab** to complete this task.
- Your submission must be a **single PDF file** that includes:
  - The source code (in plain text)
  - Screenshots of the corresponding outputs
- Name your file using the following format: **rollno_name_labtasknumber.pdf**
  Example: *25p-1234_ozair_labtask03.pdf*
- Unethical use of AI tools will result in **0 marks**.
- Late submissions will incur a penalty of **-3 marks per day**.

Tasks

ONLY USE POINTER NOTATION TO IMPLEMENT ALL THE SOLUTIONS. ARRAY INDEXING [ ] IS NOT ALLOWED.

Section 1: Data Structures

1 Convert the Existing Implementation to Store Strings

Modify the given `ArrayList` implementation so that it stores `string` values instead of integers. Update the data members, constructors, and member functions accordingly.

2 Implement remove(val)

Implement a function that removes only the **first occurrence** of the specified value from the list. The function should appropriately handle the case where the specified value does not exist in the list.

3 Implement search(val)

Implement a function that searches for the specified value in the list. The function should return `true` if the value is found and `false` otherwise.

4 Implement retrieve(val)

Implement a function that searches for the specified value and returns its index if it is found. If the value does not exist in the list, return an appropriate invalid index.

5 Implement replace(oldVal, newVal)

Implement a function that searches the entire list and replaces **all occurrences** of `oldVal` with `newVal`.

6 Implement clear()

Implement a function that removes all elements from the list, leaving it empty while maintaining the allocated array capacity.

7 Implement sort(flag)

Implement a function to sort the strings in the list:

- If `flag` is `true`, sort the list in **descending order**.
- If `flag` is `false`, sort the list in **ascending order**.

8 Implement reverse()

Implement a function that reverses the order of all elements in the list **in place**, without creating another list.

9 Implement resize() and Update add() / remove() Operations

Implement a `resize()` function to dynamically adjust the capacity of the underlying array by either growing or shrinking it as required.

- **Grow the array:** When the array becomes full, increase its capacity by **10**. The new capacity should be: `new capacity = old capacity + 10`
- **Shrink the array:** When the number of elements becomes **less than half of the current capacity**, reduce the capacity of the array to the current number of elements. The new capacity should be: `new capacity = size`
- The `resize()` function should allocate a new array with the updated capacity, copy the existing elements into it, release the old array, and update the array pointer and capacity accordingly.
- Modify the `add()` and `remove()` functions as necessary so that `resize()` is automatically called when the array needs to grow or shrink.

10 Update and Test the main() Function

Modify the `main()` function to demonstrate and test all newly implemented operations, including adding, inserting, removing, searching, retrieving, replacing, clearing, sorting, and reversing string values.

Section 2: Problem Solving

1 Search Insert Position

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order in logarithmic time.

Examples:

```
Input: nums = [1,3,5,6], target = 5
Output: 2

Input: nums = [1,3,5,6], target = 2
Output: 1

Input: nums = [1,3,5,6], target = 7
Output: 4
```

2 Plus One

You are given a large integer represented as an integer array digits, where each digits[i] is the ith digit of the integer. The digits are ordered from most significant to least significant in left-to-right order. The large integer does not contain any leading 0's.

Increment the large integer by one and return the resulting array of digits.

Examples:

```
Input: digits = [1,2,3]
Output: [1,2,4]
Explanation: The array represents the integer 123. Incrementing by one gives
123 + 1 = 124. Thus, the result should be [1,2,4].

Input: digits = [4,3,2,1]
Output: [4,3,2,2]

Input: digits = [9]
Output: [1,0]
```